

Projeto N1-4 Editor de texto - Fase II

Equipe: Carolina Gusmão, Guilherme Cordovil, Sabrina Bernardi

Entendimento do Problema e Arquitetura da Solução: Editor de Texto com Histórico Dinâmico

1. O Problema e o Impacto na Usabilidade

No desenvolvimento de aplicações modernas, como editores de texto e plataformas de código, a experiência do usuário está diretamente ligada à segurança psicológica de poder errar. Erros de digitação ou apagamentos acidentais são inevitáveis, e a incapacidade de reverter essas ações gera frustração e perda de dados. Do ponto de vista de usabilidade e negócios, as funcionalidades de Desfazer (Undo) e Refazer (Redo) são recursos críticos.

Para que essa navegação temporal funcione, o sistema precisa gerenciar de forma inteligente o histórico de estados do texto. O comportamento natural dessa mecânica exige que a última ação realizada seja a primeira a ser desfeita, obedecendo estritamente ao princípio LIFO (Last In, First Out). Como um editor recebe um número imprevisível de interações, o uso de estruturas estáticas (vetores de tamanho fixo) é ineficiente, podendo causar desperdício de memória ou, no pior dos casos, o esgotamento do limite de histórico (estouro de pilha).

2. A Proposta de Solução e Estrutura de Dados

Para atender aos requisitos de escalabilidade e otimizar os recursos do sistema, a solução foi desenvolvida em C, utilizando a estrutura de dados de Pilhas Dinâmicas implementadas através de Listas Encadeadas Simples com Alocação Dinâmica de Memória.

O sistema não registra apenas a tecla pressionada, mas adota a abordagem de *Snapshots* (padrão de projeto Memento). A cada mudança significativa, o estado completo do texto é salvo. Para gerenciar as viagens no tempo (passado e futuro), o sistema utiliza duas pilhas distintas:

- Pilha de Histórico (`topo_historico`): Armazena os estados passados do texto.
- Pilha de Refazer (`topo_refazer`): Armazena os estados futuros caso o usuário desfça uma ação e queira voltar atrás.

3. Arquitetura Técnica e Fluxo de Operações

- Estrutura do Nó: Cada nó da pilha alocado dinamicamente contém uma cópia exata do texto (`char estado[MAX_TEXTO]`) naquele momento específico e um ponteiro para o nó anterior.

- Alocação Dinâmica (**malloc** e **free**): Garante que a memória RAM do computador seja consumida sob demanda apenas quando um novo estado for gerado, devolvendo o espaço ao sistema operacional instantaneamente quando uma ação é esquecida ou a aplicação é encerrada.
- Ciclo de Vida das Ações:
 1. Inserção ou Deleção: Antes de alterar o texto atual, o sistema faz um *Push* do texto no **topo_historico**. A modificação é então aplicada. Imediatamente, a pilha de **topo_refazer** é completamente esvaziada, pois uma nova ação reescreve o "futuro" disponível.
 2. Desfazer (Undo): O sistema salva o texto atual na pilha de **topo_refazer**. Em seguida, faz um *Pop* do **topo_historico**, extraíndo o último estado salvo e substituindo o texto atual por ele.
 3. Refazer (Redo): Funciona de forma espelhada ao Undo. O estado atual é jogado para o **topo_historico**, e o sistema faz um *Pop* da pilha de **topo_refazer**, restaurando o texto que havia sido desfeito.

4. Conclusão

Esta abordagem garante que o rastreamento das ações seja absolutamente íntegro e à prova de falhas. O uso de ponteiros e alocação dinâmica faz com que o consumo de memória escale de forma sustentável, entregando um editor de texto eficiente, sem travamentos e com uma experiência de usuário confiável.